Part I: Contents

- Part I: Contents
- Part I: Figures
- Part I: Tables
- Part I: Examples
- Preface to OCP Java SE 21 Developer (Exam 1Z0-830)
  - About Part I: OCP Java SE 21 Developer
    - *Exam Objectives*
    - *Java SE Platform API Documentation*
  - Website for Part I: OCP Java SE 21 Developer
  - Request for Feedback
  - About the Authors
    - *Khalid A. Mughal*
    - *Vasily A. Strelnikov*
  - Acknowledgments
- 1 Pattern Matching for Java Objects
  - 1.1 Type Pattern Matching with the `instanceof` Pattern Match Operator
    - The `instanceof` Type Comparison Operator
    - The `instanceof` Pattern Match Operator
    - Valid Operand Types in the `instanceof` Pattern Match Operator
      - *Same Types as Operands*
      - *Unrelated Types as Operands*
      - *Unrelated Classes as Operands*
      - *Peer Classes as Operands*
      - *Final Classes as Operands*
      - *Sealed Types as Operands*
      - *Generic Types as Operands*
    - Properties of a Pattern Variable
    - Flow Sensitive Scope of Pattern Variables
  - 1.2 Type Pattern Matching with the Enhanced `switch` Construct
    - Using Type Patterns in `case` Labels
    - The `null` Value as Case Constant in `switch` Construct
    - Guarded Case Patterns
    - Case Label Dominance

Flow Sensitive Scope of a Pattern Variable in `switch` Construct
Exhaustiveness of Case Pattern Labels
Sealed Types and Exhaustiveness
Illegal Fall-Through to a Pattern in `switch` Statement
1.3 Record Patterns
1.4 Record Pattern Matching with the `instanceof` Pattern Match Operator
Type Inference of Pattern Variables using `var`
Flow Sensitive Scope of Pattern Variables in Record Patterns
Nested Record Patterns
Using Record Patterns with Generic Types
1.5 Record Pattern Matching with the Enhanced `switch` Construct
Nested Record Patterns in `case` Labels
Guarded Record Patterns in `case` Labels
Flow Sensitive Scope of Pattern Variables in the `switch` Construct
Type Inference of Record Pattern Variables in `case` Labels
Case Label Dominance in Record Patterns
Illegal Fall-Through in `case` Labels with Record Patterns
Using Sealed Types with Record Patterns in `case` Labels
Using Generic Record Patterns in `case` Labels
Using Generic Record Patterns parameterized with Sealed Types
1.6 Summary of Java Patterns
1.7 Using Qualified Name for Enum Constants in the `switch` Construct
1.8 Unexpected Failure in Pattern Matching
Review Questions
2 Sequenced Collections and Maps
2.1 Collections with Well-Defined Encounter Order
2.2 Sequenced Collections
The `SequencedCollection<E>` Interface
2.3 Sequenced Lists
The `List<E>` Interface
The `ArrayList<E>` Class
2.4 Sequenced Sets
The `SequencedSet<E>` Interface
The `SortedSet<E>` Interface
The `NavigableSet<E>` Interface
The `TreeSet<E>` Class
The `LinkedHashSet<E>` Class
2.5 Deques
The `ArrayDeque<E>` Class
The `LinkedList<E>` Class

2.6 Sequenced Maps
The SequencedMap<K, V> Interface
Views on Keys, Values, and Entries of a SequencedMap<K,V>
Composing Views
Static Entry Snapshots
The SortedMap<K,V> Interface
The NavigableMap<K,V> Interface
The TreeMap<K,V> Class
The LinkedHashMap<K,V> Class
2.7 Unmodifiable Sequenced Collections, Sets, and Maps
2.8 Sequenced Concurrent Collections
2.9 Sequenced Concurrent Maps
Review Questions
3 Virtual Threads
3.1 Motivation for Virtual Threads
3.2 Virtual Thread Execution Model
3.3 Using Thread Class to Create Virtual Threads
Logging Information during Program Execution
Creating and Starting a Virtual Thread
3.4 Using Thread Builders to Create Virtual Threads
Important Aspects of Virtual Threads
Is a Thread Virtual?
Virtual Threads are Daemon Threads
Virtual Threads have Normal Priority
Virtual Threads belong to VirtualThreads Group
3.5 Using Thread Factory to Create Threads
3.6 Using Thread Executor Services
Using the Virtual-Thread-Per-Task Executor Service
Customizing the Thread-Per-Task Executor Service
The Executors Utility Class
3.7 Scalability of Throughput with Virtual Threads
3.8 Best Practices for Using Virtual Threads
Avoid Pinning of Virtual Threads
Pinning in Synchronized Block
Avoiding Pinning with a Reentrant Lock
Avoid Using Virtual Threads for CPU-Bound Tasks
Avoid Pooling of Virtual Threads
Minimize Using Thread-Local Variables with Virtual Threads
Avoid Substituting Virtual Threads for Platform Threads
Review Questions

A Exam Topics: Java SE 21 Developer Professional
B Annotated Answers to Review Questions
C Miscellaneous API Updates
C.1 The `String` and `StringBuilder` Classes
Searching in a String
Extending a String Builder
C.2 Constructing Locales
Part I: Index

Previous chapter
Dedication Page

Next chapter
Part I: Figures